

Project: Rule-Based Language with Execution and Analysis
SI2002 Formal Languages
ST0270 Formal Languages and Compilers
CM0081 Formal Languages and Automata

Andrés Castro, César Guerra, Oscar Rodriguez, Sergio Ramírez

1 Introduction

A fundamental way to represent knowledge and automate decisions in computing is through rules of the form “*if A then B*”. This paradigm, known as rule-based systems, has been central in artificial intelligence as well as in the theory of languages and databases.

From early expert systems to modern declarative languages, the underlying idea is the same: represent knowledge using rules and derive new conclusions from known facts.

Although these systems are often studied in artificial intelligence, their implementation relies on core concepts from formal languages:

- a formal language to express rules
- a grammar that defines its syntax
- a parser that builds a structured representation
- a semantics that defines how rules are evaluated

In this sense, building a rule-based system follows the same conceptual path as a compiler: from syntactic analysis to semantic execution.

2 Project Description

In this project you will develop a system to define, process, and execute a rule-based language.

The system must include:

- Lexical analysis
- Syntactic analysis (LL(1) or LR(1))
- Construction of an *Abstract Syntax Tree* (AST)
- An interpreter for the language
- A basic static analysis module

2.1 Language

The language consists of rules of the form:

```
rule <id>:  
if <condition> then <action>
```

Allowed conditions:

- Comparisons: $id > n$, $id < n$, $id = n$
- Identifiers (true if the fact is active)
- Conjunctions with AND

2.2 Minimal Grammar Specification

All implementations must support at least the following grammar over the alphabet

$$\Sigma = \{\text{rule, if, then, AND, :, >, <, =, id, value}\}.$$

```
Program → RuleList  
RuleList → Rule RuleList |  $\varepsilon$   
Rule → rule id : if Cond then Action  
Cond → Cond AND Cond | Atom  
Atom → id RelOp value | id  
RelOp → > | < | =  
Action → id
```

Here, `id` denotes identifiers and `value` denotes integer literals.

Notes:

- Identifiers (`id`) represent variables or facts.
- Values (`value`) are integers.
- The operator `AND` is the only logical connective required.
- Parentheses are not required.
- This grammar is not LL(1) as written. Groups choosing LL(1) must transform it accordingly.
- Groups choosing LR(1) may use the grammar directly.

2.3 Input Format Specification

A program consists of two parts:

- A set of rules
- An optional initial state

Rules. Rules must follow exactly the syntax defined by the grammar. Each rule must be written using the format:

```
rule <id>:
  if <condition> then <action>
```

Rules are separated by one or more blank lines.

Initial State. The initial state is provided separately from the program and consists of:

- Variable assignments:

```
id = integer
```

- Active facts:

```
id
```

Each element must appear on a separate line.

Constraints.

- Identifiers consist of lowercase letters, digits, and underscores, and must start with a letter.
- Integers are base-10 and may be positive or zero.
- Keywords (**rule**, **if**, **then**, **AND**) are case-sensitive.
- Extra whitespace (spaces and blank lines) should be ignored.

2.4 Canonical AST Structure

All implementations must construct an Abstract Syntax Tree (AST) that is structurally equivalent to the following specification.

The AST must represent the program using the following node types:

- **Program**
 - A list of rules
- **Rule**
 - Identifier (rule name)
 - Condition
 - Action
- **Condition**
 - Either:

- * A conjunction of two conditions (AND)
- * An atomic condition
- **Atomic Condition**
 - Either:
 - * A comparison: (identifier, operator, value)
 - * A fact: (identifier)
- **Action**
 - Identifier (fact to activate)

2.5 Canonical Evaluation Strategy

The execution of a program is defined over an *environment* (or state), which consists of:

- A mapping from identifiers to integer values
- A set of active facts (identifiers)

Identifiers may appear either as variables (with associated values) or as facts. These roles should be treated independently.

Initial State. The initial state is given as input:

- Variable assignments (e.g., `temp = 35`)
- Initially active facts (optional)

Evaluation of Conditions. Conditions are evaluated as follows:

- $\text{Cmp}(x, op, v)$ is true if the comparison between the value of x and v holds
- $\text{Fact}(x)$ is true if x is an active fact
- $\text{And}(c_1, c_2)$ is true if both c_1 and c_2 are true

Rule Application. A rule (id, c, a) is applicable if its condition c evaluates to true.

When a rule is applied:

- Its action $\text{Act}(x)$ activates the fact x

Execution Model (Fixed Point). The program is evaluated iteratively as follows:

1. Initialize the set of active facts with the initial facts
2. Repeat:
 - Evaluate all rules
 - Collect all actions whose rules are applicable
 - Add the resulting facts to the set of active facts (without duplicates)
3. Until no new facts are added (fixed point)

Determinism. The evaluation must be independent of the order in which rules are processed.

2.6 Canonical Output Format

The output of a program consists of the set of facts activated during execution.

Format.

- Each activated fact must be printed on a separate line.
- Each fact must be printed at most once.
- The output must contain only the fact identifiers (no additional text).

Ordering. The output must be sorted in lexicographic (alphabetical) order.

Empty Output. If no facts are activated, the output must be exactly:

(no output)

Static Analysis Messages. Static analysis results (e.g., conflicts, redundancies) must be printed after the execution output.

Each message must be printed on a separate line using the following formats:

- Conflict:

Action <id> generated by r1, r2, ...

- Redundancy:

Redundant rules: r1, r2

- Potentially inactive rule:

Potentially inactive rule: <id>

3 Assignment Submission

You should follow the instructions below for this assignment.

1. It is allowed to work in groups of two or three students.
2. A README.md file (Markdown format) in English is required. It must contain at least the following information:
 - Full names of group members.

- Versions of the operating system, programming language, and tools used in your implementation.
 - Detailed instructions for running your implementation.
3. Do not submit unnecessary files or directories.
 4. No additional language features (e.g., OR, NOT, parentheses) should be introduced in the base implementation.
 5. Submissions must be made through Interactiva. Each group must be registered on the platform prior to submission.

4 Deadline

To be defined in each group. Tentative: week 17 or 18.

5 Grading

- Oral presentation: at least 70%
- Submission (code, documentation, tests): remaining percentage

Exact distribution will be defined for each group.

During the oral presentation, students may be asked to manually trace the evaluation of a program.

6 Guidelines

- Any programming language may be used.
- Parsers must be implemented by the students. The use of parser generators or external parsing libraries is not allowed.
- Code must be properly documented with comments.
- The implementation should follow a modular design (e.g., clear separation between lexer, parser, AST, and interpreter components).

7 Test Cases

7.1 Case 1

```
rule r1:
    if temp > 30 then alert
```

State:

```
temp = 35
```

Output:

```
alert
```

7.2 Case 2

```
rule r1:  
    if temp > 30 then alert
```

```
rule r2:  
    if alert then fan_on
```

State:

temp = 35

Output:

```
alert  
fan_on
```

7.3 Case 3

```
rule r1:  
    if temp > 30 then alert
```

State:

temp = 20

Output:

(no output)

7.4 Case 4

```
rule r1:  
    if temp > 30 AND humidity < 50 then alert
```

State:

temp = 35

humidity = 40

Output:

```
alert
```

7.5 Case 5

State:

temp = 35

humidity = 60

Output:

(no output)

7.6 Case 6

```
rule r1:  
    if temp > 30 then fan_on
```

```
rule r2:  
    if humidity < 50 then fan_on
```

State:

```
temp = 35  
humidity = 40
```

Output:

```
fan_on
```

Analysis:

Action fan_on generated by r1, r2

7.7 Case 7

```
rule r1:  
    if temp > 30 then alert
```

```
rule r2:  
    if temp > 30 then alert
```

Analysis:

Redundant rules: r1, r2

7.8 Case 8

```
rule r1:  
    if temp > 30 then alert
```

```
rule r2:  
    if alert then fan_on
```

```
rule r3:  
    if humidity < 20 then cooling
```

State:

```
temp = 25  
humidity = 50
```

Analysis:

Potentially inactive rule: r3

Additional Test Case (for grading)

```
rule r1:  
  if a then b
```

```
rule r2:  
  if b then c
```

```
rule r3:  
  if c then d
```

```
rule r4:  
  if d then e
```

State:

a

Output:

b

c

d

e